```
# architect_gemini.py
import ast
import json
import os
import subprocess
import sys
from file_manager import register_file, load_index

TASKS_FILE = "tasks.json"

# Appended to every code-generation task so small models can't miss it.
CODE_CONSTRAINTS = (
    "\n\nHard requirements: the script must finish on its own in under 10
seconds. "
    "Do NOT use `while True` or any infinite loop. Use a `for` loop with a
fixed "
    "iteration count (e.g. 50). Do NOT use input(). Do NOT write files. "
    "If you use time.sleep, keep it at 0.05 seconds or less. "
    "Print the results to the terminal at the end."
)


class SelfCodingAgent:
    def __init__(self, core_name="architect_gemini.py"):
        self.core_name = core_name
        register_file(self.core_name, "Main autonomous agent core logic with
local Qwen-Coder integration and self-repair.")
        self.init_tasks()

    def init_tasks(self):
        """Ensures the task queue file exists."""
        if not os.path.exists(TASKS_FILE):
            default_data = {
                "tasks": [
                    {"id": 1, "title": "Self-Introspection Audit",
"description": "Read own source code and log structural summary.", "status":
"pending"},
                    {"id": 2, "title": "Generate Utility Module",
"description": "Write a Python script that calculates rolling moving averages
for data telemetry.", "status": "pending"}
                ]
            }
```

```
            with open(TASKS_FILE, "w") as f:
                json.dump(default_data, f, indent=4)
            register_file(TASKS_FILE, "Persistent task queue for agent
objectives.")

    def load_tasks(self):
        with open(TASKS_FILE, "r") as f:
            return json.load(f)

    def save_tasks(self, data):
        with open(TASKS_FILE, "w") as f:
            json.dump(data, f, indent=4)

    def read_own_source(self):
        """Introspects its own source code."""
        if not os.path.exists(self.core_name):
            return None
        with open(self.core_name, "r") as f:
            return f.read()

    def ask_qwen_coder(self, task_description):
        """Invokes Qwen2.5-Coder locally via llama-cli non-interactively to
generate code and reasoning."""
        cmd = [
            "llama-cli",
            "-hf", "Qwen/Qwen2.5-Coder-1.5B-Instruct-GGUF:Q4_K_M",
            "-p", f"You are an autonomous Python developer agent. Write a
clean, standalone Python script for the task below. Provide a brief
explanation of how it works, followed by the code block.\n\nTask:
{task_description}",
            "-n", "768",
            "--gpu-layers", "0",
            "-st",                 # Single-turn exit flag
            "--no-display-prompt"  # Suppress prompt echo
        ]

        print("\n[AI AGENT] Consulting local Qwen2.5-Coder...")
        try:
            result = subprocess.run(
                cmd,
                capture_output=True,
                text=True,
                stdin=subprocess.DEVNULL,  # Force immediate EOF so it never
hangs on input
                # llama-cli writes its chat output to /dev/tty when it has
```

```
one, which
                # bypasses capture_output. Detaching from the terminal forces
stdout.
                start_new_session=True,
                timeout=900
            )
            if result.returncode == 0:
                text = result.stdout.strip()
                if not text:
                    print("[AI WARNING] llama-cli exited cleanly but returned
no text.")
                return text
            else:
                print(f"[AI ERROR] llama-cli error: {result.stderr}")
                return None
        except Exception as e:
            print(f"[AI EXCEPTION] {e}")
            return None

    def extract_code_block(self, raw_output):
        """Pulls the python code out of the model's markdown output (no
validation)."""
        code_block = raw_output

        if "```python" in raw_output:
            parts = raw_output.split("```python")
            if len(parts) > 1:
                code_block = parts[1].split("```")[0].strip()
        elif "```" in raw_output:
            parts = raw_output.split("```")
            if len(parts) > 1:
                code_block = parts[1].strip()

        return code_block

    def extract_and_validate_code(self, raw_output):
        """Extracts the code block, checks syntax, and rejects infinite
loops."""
        code_block = self.extract_code_block(raw_output)

        try:
            tree = ast.parse(code_block)
        except SyntaxError as e:
            return False, f"Syntax Error: {e}"
```

```
        # Reject `while True` / `while 1` before wasting a 15 second health
check.
        for node in ast.walk(tree):
            if (
                isinstance(node, ast.While)
                and isinstance(node.test, ast.Constant)
                and node.test.value  # True, 1, or any other truthy constant
            ):
                return False, (
                    "Found an infinite `while True` loop. Replace it with a
`for` loop "
                    "that runs a fixed number of iterations (for example
50)."
                )

        return True, code_block

    def execute_and_verify_detailed(self, filename):
        """Runs a python script as a subprocess and returns (success_bool,
error_message)."""
        print(f"[HEALTH CHECK] Executing {filename}...")
        try:
            result = subprocess.run(
                [sys.executable, filename],
                capture_output=True,
                text=True,
                stdin=subprocess.DEVNULL,  # input() fails fast instead of
hanging
                timeout=15,
            )
        except subprocess.TimeoutExpired:
            print("[HEALTH CHECK FAIL] Timed out after 15 seconds")
            return False, (
                "The script never exited (it ran for 15 seconds before being
killed). "
                "Remove every `while True` loop. Use a `for` loop with a
fixed number of "
                "iterations (for example 50) and at most time.sleep(0.05)."
            )
        except Exception as e:
            print(f"[HEALTH CHECK EXCEPTION] {e}")
            return False, str(e)

        if result.returncode == 0:
            print(f"[HEALTH CHECK PASS] Output:\n{result.stdout.strip()}")
```

```
            return True, ""

        print(f"[HEALTH CHECK FAIL] Error:\n{result.stderr.strip()}")
        return False, result.stderr.strip()

    def generate_with_repair(self, task_title, task_description, filename,
max_attempts=3):
        """Attempts to generate, validate, and self-repair code using Qwen-
Coder based on feedback."""
        base_task = task_description + CODE_CONSTRAINTS
        current_prompt = base_task

        for attempt in range(1, max_attempts + 1):
            print(f"\n[ATTEMPT {attempt}/{max_attempts}] Generating code for:
{task_title}")
            raw_output = self.ask_qwen_coder(current_prompt)
            if not raw_output:
                continue

            print("\n--- MODEL EXPLANATION & OUTPUT ---")
            print(raw_output)
            print("----------------------------------\n")

            # Extract and validate (syntax + no infinite loops)
            ok, code_or_err = self.extract_and_validate_code(raw_output)
            if not ok:
                print(f"[VALIDATION FAIL] Attempt {attempt}: {code_or_err}")
                print("Feeding error back to model...")
                bad_code = self.extract_code_block(raw_output)
                current_prompt = (
                    f"{base_task}\n\nThis
code:\n```python\n{bad_code}\n```\n"
                    f"was rejected: {code_or_err}\n"
                    "Return the fully corrected standalone script in one
```python block."
                )
                continue

            # Write candidate file
            with open(filename, "w") as f:
                f.write(code_or_err)

            # Run runtime health check
            passed, err_msg = self.execute_and_verify_detailed(filename)
            if passed:
```
```

```
                print(f"[SUCCESS] Module {filename} passed all health checks
on attempt {attempt}!")
                register_file(filename, f"Autonomously generated via Qwen
for: {task_title}")
                return True

            print(f"[RUNTIME FAIL] Attempt {attempt} failed execution.
Feeding error back to model...")
            current_prompt = (
                f"{base_task}\n\nThis code:\n```python\n{code_or_err}\n```\n"
                f"failed at runtime with this error:\n{err_msg}\n"
                "Return the fully fixed, corrected standalone script in one
```python block."
            )

        print(f"[ERROR] Max repair attempts reached for task: {task_title}")
        return False

    def process_next_task(self):
        data = self.load_tasks()
        pending_tasks = [t for t in data["tasks"] if t["status"] ==
"pending"]

        if not pending_tasks:
            print("[QUEUE] No pending tasks remaining.")
            return

        task = pending_tasks[0]
        print(f"\n[QUEUE] Processing Task #{task['id']}: {task['title']}")
        print(f"Details: {task['description']}")

        if task['id'] == 1:
            source = self.read_own_source()
            if source:
                print(f"[INTROSPECTION] Read {len(source)} chars from
{self.core_name}.")
                task['status'] = "completed"
        else:
            # Generate clean filename based on task title
            target_filename = task['title'].lower().replace(" ", "_") + ".py"

            # Let the agent build and self-repair
            success = self.generate_with_repair(task['title'],
task['description'], target_filename)
            if success:
```
```

```
                task['status'] = "completed"

        self.save_tasks(data)

    def inspect_state(self):
        index = load_index()
        print("\n--- AGENT FILE INDEX ---")
        for fname, meta in index["files"].items():
            print(f"• {fname} (v{meta['version']}): {meta['description']}")
        print("------------------------")

        data = self.load_tasks()
        print("\n--- TASK QUEUE STATUS ---")
        for t in data["tasks"]:
            status_icon = "✓" if t['status'] == "completed" else "•"
            print(f"{status_icon} [{t['status'].upper()}] Task #{t['id']}:
{t['title']}")
        print("-------------------------\n")


if __name__ == "__main__":
    agent = SelfCodingAgent()
    agent.inspect_state()
    agent.process_next_task()
    agent.inspect_state()
```